IT STUDY HUB

COMPLETE C++ PROGRAMMING SERIES

MODULE 3: CONTROL FLOW & FUNCTIONS

Authors: Mithila Rani Saha & Dithsa Dutta

Website: www.itstudyhub.dpdns.org

Module Overview

1. About This Module

Welcome to **Module 3: Control Flow & Functions** in the IT Study Hub Complete C++ Programming Series. Having mastered basic variables, data types, and operators in Modules 1 and 2, you are now ready to teach your programs how to

make decisions, repeat tasks efficiently, and organize logic into reusable modular units. Authored by Mithila Rani Saha and Dithsa Dutta, this module establishes the algorithmic backbone required for professional software engineering.

2. Learning Outcomes

By completing this module, you will be able to:

- Control program execution using conditional branching (`if`, `else-if`, `else`, nested conditions, and `switch`).
- Automate repetitive operations using `for`, `while`, and `do-while` loops with nested iterations.
- Manage loop flows precisely using `break`, `continue`, and control jump statements like `goto`.
- Design modular applications utilizing function declarations, definitions, parameters, return values, and default arguments.
- Implement advanced function concepts including function overloading, inline expansion, recursion, and pass-by-reference mechanics.

3. Module Coverage

This module comprehensively covers:

- **Conditional Branching:** Simple `if`, `if-else`, ladders, nested conditions, and `switch` dispatcher.
- **Iterative Loops:** `for`, `while`, `do-while`, nested loops, and jump controls (`break`, `continue`, `goto`).
- **Modular Functions:** Signatures, arguments, return mechanics, default parameters, and overloading.
- **Advanced Functions:** Inline functions, recursive problem solving, pass-by-value vs pass-by-reference, and `const` parameters.

- **Hands-On Project:** Building a comprehensive Menu-Driven C++ Calculator & Number Tool.
- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 15+ technical interviews, and coding challenges.

IT STUDY HUB

4. Main Notes: Theoretical Foundations

Control structures and functions dictate how instructions are sequenced and encapsulated. Below are the exhaustive foundational notes for Module 3.

Concept: Conditional Branching: if, if-else, else-if

Simple Explanation: Conditional statements allow a program to evaluate boolean expressions and execute specific blocks of code only when those conditions evaluate to true. The `if` statement handles single checks, `if-else` adds an alternative branch, and `else-if` creates multi-way decision ladders.

> ***Real-Life Analogy:*** *Think of conditional branching as a highway toll booth decision: IF you have an electronic tag, you go through the express lane; ELSE IF you have exact coins, you use the automated coin basket; ELSE you must drive to the manual teller window.*

```
Syntax:
if (condition) {
// statements
} else if (condition2) {
// statements
} else {
// statements
}
```

```
#include
using namespace std;

int main() {
    int score = 85;

    if (score >= 90) {
        cout << "Grade: A" << endl;
```

```
    } else if (score >= 75) {
        cout << "Grade: B" << endl;
    } else {
        cout << "Grade: C" << endl;
    }
    return 0;
}
```

Output:
```
Grade: B
```

Line-by-Line Breakdown:

- **if (score >= 90)**: Evaluates whether score is 90 or above (false here).
- **else if (score >= 75)**: Evaluates whether score is 75 or above (true here, executes block).
- **else**: Fallback default block if all previous conditions fail.

Real-World Application: Authentication systems evaluate user credentials (`if password_match`) to grant access or trigger error handling (`else`).

Common Mistakes: Placing a semicolon immediately after the `if` condition parenthesis (`if (x > 5);`), which prematurely terminates the conditional block.

Best Practices: Order your `else-if` conditions logically from most specific to least specific to avoid masking matching criteria.

Actionable Practice: Write an `if-else` chain that categorizes a person's age into Child, Teenager, Adult, or Senior.

Concept: Nested Conditions and Switch Statement

Simple Explanation: Nested conditions place `if` statements inside other `if` blocks for hierarchical decision-making. The `switch` statement evaluates a single integer, character, or enumerated expression against multiple discrete `case` labels, providing a cleaner alternative to long `else-if` ladders.

> ***Real-Life Analogy:*** *A nested condition is like opening security doors inside a vault where each door requires a separate key. A `switch` statement is like an elevator panel where pressing button '3' instantly routes you directly to floor 3 without checking every floor individually.*

```
Syntax:
switch (variable) {
case 1:
// code
break;
default:
// code
}
```

```
#include
using namespace std;

int main() {
    int day = 3;
    switch(day) {
        case 1: cout << "Monday"; break;
        case 2: cout << "Tuesday"; break;
        case 3: cout << "Wednesday"; break;
        default: cout << "Invalid Day";
    }
    cout << endl;
    return 0;
}
```

```
Output:
Wednesday
```

Line-by-Line Breakdown:

- **switch(day)**: Evaluates the integer value of `day` (3).
- **case 3:**: Matches `day == 3`, executes `cout << "Wednesday"`.
- **break;**: Exits the switch block immediately to prevent fall-through.

Real-World Application: Menu-driven CLI applications use `switch` statements to handle user command selections efficiently.

Common Mistakes: Forgetting the `break` statement in a `switch` case, causing 'fall-through' where execution spills into subsequent cases unintentionally.

Best Practices: Always include a `default` case in every `switch` block to gracefully handle unexpected inputs.

Actionable Practice: Write a nested `if` condition that checks if a number is positive, and if so, whether it is even or odd.

Concept: Loops: for, while, and do-while

Simple Explanation: Loops automate repetitive tasks. The `for` loop is ideal when the iteration count is known in advance. The `while` loop checks a condition before executing the block. The `do-while` loop executes the code block at least once before checking the condition at the end.

> ***Real-Life Analogy:*** *A `for` loop is like running exactly 5 laps around a track. A `while` loop is like drinking water while you are still thirsty. A `do-while` loop is like tasting a dish first to see if you like it before deciding whether to eat more.*

Syntax:
```
for (init; condition; update) { }
while (condition) { }
do { } while (condition);
```

```
#include
using namespace std;

int main() {
    // For loop
    for(int i = 1; i <= 3; i++) {
        cout << "For i: " << i << endl;
    }

    // While loop
    int w = 1;
    while(w <= 2) {
        cout << "While w: " << w << endl;
        w++;
    }
    return 0;
}
```

Output:
```
For i: 1
For i: 2
For i: 3
While w: 1
While w: 2
```

Line-by-Line Breakdown:

- **for(int i = 1; i <= 3; i++)**: Initializes `i=1`, runs while `i<=3`, increments `i` each iteration.

- **while(w <= 2)**: Evaluates condition before running the loop body.

Real-World Application: Game loops run continuously using `while` structures, updating physics and rendering frames until the user quits.

> **Common Mistakes:** Creating infinite loops by forgetting to update the loop control variable (e.g., omitting `i++`).

> **Best Practices:** Use `for` loops when the exact number of iterations is known, and `while`/ `do-while` when looping depends on dynamic runtime conditions.

> **Actionable Practice:** Write a `do-while` loop that repeatedly prompts the user to enter a positive number until they comply.

Concept: Nested Loops and Jump Statements (break, continue, goto)

Simple Explanation: Nested loops place one loop inside another, creating multidimensional execution grids. Jump statements alter normal execution flow: `break` terminates a loop or switch entirely, `continue` skips the current iteration and jumps to the next, and `goto` jumps unconditionally to a labeled statement.

> ***Real-Life Analogy:*** *Nested loops are like the hands of a clock (minutes hand completing a full circle for every single hour). `break` is like pulling an emergency brake, while `continue` is skipping one track on a playlist and moving to the next.*

```
Syntax:
break;
continue;
label:
```

```
// code
goto label;
```

```
#include
using namespace std;

int main() {
    for(int i = 1; i <= 3; i++) {
        if(i == 2) continue; // Skips iteration when i is 2
        cout << "Count: " << i << endl;
    }
    return 0;
}
```

```
Output:
Count: 1
Count: 3
```

Line-by-Line Breakdown:

- **if(i == 2) continue;**: When `i` equals 2, skips the print statement and jumps straight to `i++`.
- Output shows 'Count: 1' and 'Count: 3', omitting 2.

Real-World Application: Matrix multiplication algorithms rely heavily on nested loops to traverse rows and columns.

Common Mistakes: Overusing `goto` statements, which creates tangled, unmaintainable 'spaghetti code'.

Best Practices: Avoid `goto` entirely in modern C++ software engineering, as structured loops and functions provide cleaner alternatives.

Actionable Practice: Write nested loops to print a 3x3 grid of asterisks (`*`).

Concept: Functions: Declaration, Definition, Parameters, and Return Values

Simple Explanation: A function is a self-contained block of reusable code that performs a specific task. Function declaration (prototype) informs the compiler of the function's signature, while the definition contains the actual executable body. Parameters receive input arguments, and `return` sends output back to the caller.

> ***Real-Life Analogy:*** *A function is like a specialized kitchen blender. You give it fruit chunks (parameters), it processes them inside the machine (definition), and it pours out a smoothie (return value).*

```
Syntax:
return_type functionName(parameter_list) {
// body
return value;
}
```

```
#include
using namespace std;

// Declaration / Prototype
int addNumbers(int a, int b);

int main() {
    int sum = addNumbers(10, 20);
    cout << "Sum: " << sum << endl;
    return 0;
}

// Definition
int addNumbers(int a, int b) {
    return a + b;
}
```

```
Output:
Sum: 30
```

Line-by-Line Breakdown:

- **int addNumbers(int a, int b);**: Prototype declaring that a function named `addNumbers` takes two ints and returns an int.
- **addNumbers(10, 20)**: Calls the function, passing arguments 10 and 20.
- **return a + b;**: Computes and returns 30 to `main()`.

Real-World Application: Large software systems are divided into thousands of specialized functions (e.g., `renderGraphics()`, `calculateTax()`, `connectDatabase()`).

Common Mistakes: Forgetting to match the return type of the function definition with the type declared in the function prototype.

Best Practices: Keep functions small and focused on a single responsibility (Single Responsibility Principle).

Actionable Practice: Write a function that calculates and returns the cube of a number.

Concept: Advanced Functions: Default Arguments and Overloading

Simple Explanation: Default arguments allow callers to omit certain arguments, automatically falling back to predefined default values. Function overloading enables multiple functions to share the exact same name as long as their parameter signatures (types or number of parameters) are distinct.

> ***Real-Life Analogy:*** *Default arguments are like ordering a burger that comes with cheese by default unless specified otherwise. Function overloading is like a multi-tool knife blade that cuts wood if you push hard, or paper if you push lightly—same tool name, different behavior based on inputs.*

```
Syntax:
void printBox(int width, int height = 10);
int compute(int x);
double compute(double x);
```

```
#include
using namespace std;

// Overloaded functions
int multiply(int a, int b) { return a * b; }
double multiply(double a, double b) { return a * b; }

int main() {
    cout << "Int multiply: " << multiply(4, 5) << endl;
    cout << "Double multiply: " << multiply(2.5, 3.0) << endl;
    return 0;
}
```

```
Output:
Int multiply: 20
Double multiply: 7.5
```

Line-by-Line Breakdown:

- Two functions named `multiply` exist simultaneously; the compiler distinguishes them by checking whether arguments passed are `int` or `double`.

Real-World Application: A `print()` function can be overloaded to accept strings, integers, or floating-point numbers seamlessly.

Common Mistakes: Overloading functions based solely on their return type while keeping parameter lists identical (causes compilation error).

Best Practices: Use default arguments for optional configuration settings rather than creating dozens of redundant overloaded function signatures.

Actionable Practice: Create two overloaded `calculateArea()` functions: one for a rectangle (length, width) and one for a circle (radius).

Concept: Inline Functions and Recursion

Simple Explanation: An `inline` function instructs the compiler to insert the function's complete body directly into every place it is called, eliminating function call overhead. Recursion is a technique where a function calls itself directly or indirectly to solve smaller instances of the same problem.

Real-Life Analogy: *An inline function is like having a shortcut blueprint pasted directly into your house plans rather than walking over to consult a separate manual. Recursion is like nesting Russian Matryoshka dolls, where each doll opens to reveal a smaller identical doll until you reach the smallest base doll.*

```
Syntax:
inline int square(int x) { return x * x; }

int factorial(int n) {
if (n <= 1) return 1; // Base case
return n * factorial(n - 1);
}
```

```
#include
using namespace std;

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int main() {
    cout << "Factorial of 5: " << factorial(5) << endl;
    return 0;
}
```

Output:
```
Factorial of 5: 120
```

Line-by-Line Breakdown:

- **if (n <= 1) return 1;**: The critical base case that stops infinite recursion.
- **n * factorial(n - 1)**: Recursive step breaking the problem down into smaller sub-problems.

Real-World Application: Recursion is used in file system directory traversal, tree algorithms (DOM parsing), and quicksort sorting.

Common Mistakes: Omitting the base case in a recursive function, leading to infinite recursion and a stack overflow crash.

Best Practices: Only use `inline` for very small, frequently executed utility functions, as excessive inlining bloats executable binary size.

Actionable Practice: Write a recursive function to compute the Nth Fibonacci number.

Concept: Parameter Passing: Pass by Value vs Pass by Reference & Const

Simple Explanation: Pass by value copies the argument's value into the function, protecting the original variable from modification. Pass by reference (`&`) passes the memory address of the variable, allowing the function to modify the original variable directly. Adding `const` protects referenced data from accidental modification.

> ***Real-Life Analogy:*** *Pass by value is like photocopying a document and handing the copy to a coworker to mark up. Pass by reference is like handing someone the original master document where any edits directly alter the main master file.*

Syntax:

```
void modifyByVal(int x);
void modifyByRef(int &x);
void readOnly(const int &x);
```

```
#include
using namespace std;

void addTen(int &num) {
    num += 10;
}

int main() {
    int val = 20;
    addTen(val);
    cout << "Modified val: " << val << endl;
    return 0;
}
```

Output:

```
Modified val: 30
```

Line-by-Line Breakdown:

- **int &num**: Receives `val` by reference. Adding 10 to `num` directly updates `val` in `main()`.

Real-World Application: Passing large objects (like 3D game models or massive strings) by `const` reference avoids costly memory copying while protecting data integrity.

Common Mistakes: Forgetting the reference ampersand `&` when attempting to modify a caller's variable inside a function.

Best Practices: Pass large structures and objects by `const &` (const reference) for high performance and safety.

Actionable Practice: Write a function that swaps two integers using pass-by-reference parameters.

IT STU

Visual Workflows & Architecture

Architectural diagrams mapping out execution paths, loop lifecycles, function call stacks, and parameter passing mechanics.

Diagram 1: if/else Decision Flow

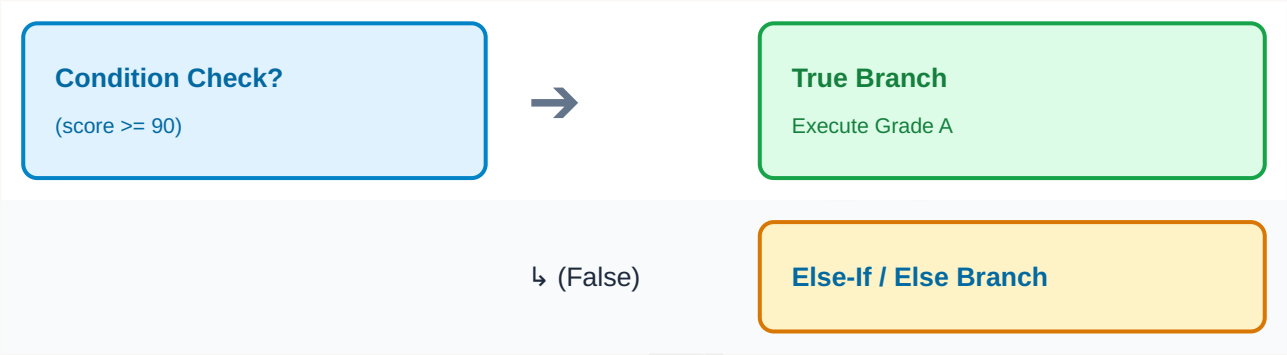


Diagram 2: Loop Lifecycles (for vs while vs do-while)

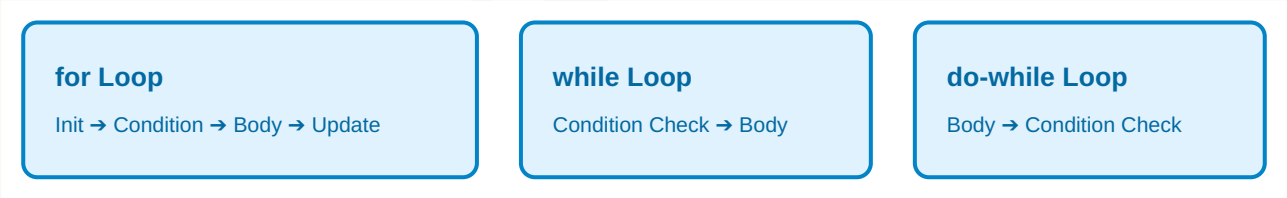


Diagram 3: Function Execution Workflow

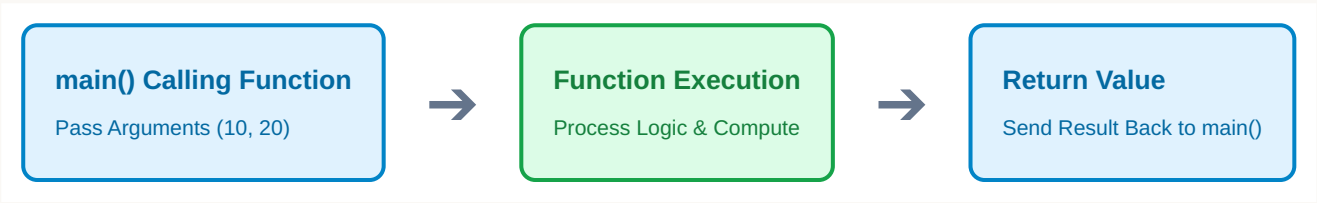
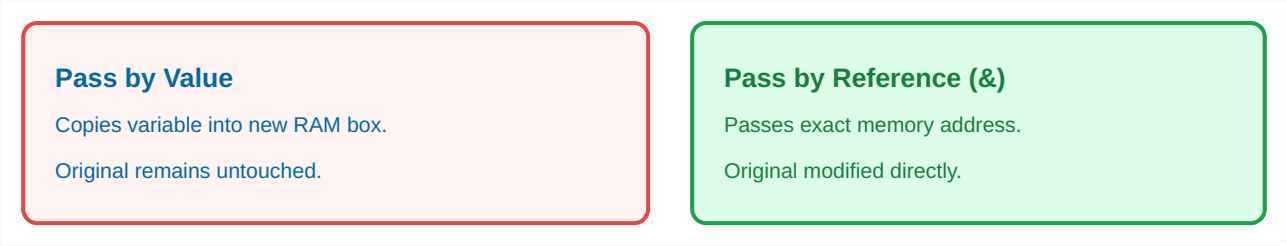


Diagram 4: Pass by Value vs Pass by Reference



IT STUDY HUB

Comparative Analysis

Professional C++ development requires choosing the exact right control structure or function mechanism for performance and readability.

Table 1: if-else vs switch

Aspect	if-else Ladder	switch Statement
Evaluation Type	Boolean expressions (ranges, comparisons like `x > 10`)	Discrete equality checks (integer/ character constants)
Performance	Slower for numerous conditions (evaluates sequentially)	Faster for many options (optimized via jump tables)
Readability	Can become cumbersome with many branches	Extremely clean and structured for menu options

Table 2: for vs while vs do-while

Loop Type	Iteration Count	Condition Check Position	Guaranteed Execution
for	Known / Fixed	Beginning (Top)	No (0 times if false)
while	Unknown / Conditional	Beginning (Top)	No (0 times if false)
do-while	Unknown / Conditional	End (Bottom)	Yes (At least 1 time)

Table 3: break vs continue

Statement	Action	Effect on Loop
break	Terminates the loop or switch entirely	Exits the loop immediately; execution continues after loop block
continue	Skips current iteration	Jumps straight to the next loop iteration check/ update

Table 4: Pass by Value vs Pass by Reference

Feature	Pass by Value	Pass by Reference
Memory Action	Copies data into new function parameter stack memory	Passes reference alias (memory address) directly
Safety	High (original variable protected)	Lower (original variable can be modified unless `const`)
Speed	Slower for large objects (due to copy overhead)	Lightning fast (no copying overhead)

Table 5: Normal Function vs Inline Function

Aspect	Normal Function	Inline Function
Compilation	Compiled as a separate subroutine block in memory	Expanded directly inline at each call site by compiler
Overhead	Incurs function call stack overhead	Zero function call overhead
Binary Size	Compact binary size	Can bloat binary size if overused

HANDS-ON PROJECT: MENU-DRIVEN C++ CALCULATOR & NUMBER TOOL

In this comprehensive hands-on project, we combine conditions, loops, switch statements, functions, parameters, return values, pass-by-reference mechanics, and recursion into a professional interactive utility suite.

Project Specification

Problem Statement: Build a menu-driven C++ application that provides 10 professional mathematical and number-theoretic tools: Addition, Subtraction, Multiplication, Division, Even/Odd check, Prime number check, Factorial, Fibonacci series, Maximum/Minimum comparison, and Exit. The program must run continuously in a loop until the user chooses to exit.

```
#include <iostream>
using namespace std;

// Function Prototypes
void showMenu();
double add(double a, double b);
double subtract(double a, double b);
double multiply(double a, double b);
double divide(double a, double b, bool &success);
bool isEven(int n);
bool isPrime(int n);
long long factorial(int n);
void fibonacci(int n);
void findMaxMin(double a, double b, double &maxOut, double &minOut);

int main() {
    int choice;
    do {
        showMenu();
        cin >> choice;

        switch(choice) {
```

```
            case 1: {
                double a, b;
                cout << "Enter two numbers: ";
                cin >> a >> b;
                cout << "Result: " << add(a, b) << endl;
                break;
            }
            case 2: {
                double a, b;
                cout << "Enter two numbers: ";
                cin >> a >> b;
                cout << "Result: " << subtract(a, b) << endl;
                break;
            }
            case 3: {
                double a, b;
                cout << "Enter two numbers: ";
                cin >> a >> b;
                cout << "Result: " << multiply(a, b) << endl;
                break;
            }
            case 4: {
                double a, b;
                bool success = true;
                cout << "Enter numerator and denominator: ";
                cin >> a >> b;
                double res = divide(a, b, success);
                if(success) cout << "Result: " << res << endl;
                break;
            }
            case 5: {
                int n;
                cout << "Enter an integer: ";
                cin >> n;
                cout << n << (isEven(n) ? " is Even." : " is Odd.")
<< endl;
                break;
            }
            case 6: {
                int n;
                cout << "Enter a positive integer: ";
                cin >> n;
                cout << n << (isPrime(n) ? " is Prime." : " is NOT
Prime.") << endl;
                break;
            }
```

```
            case 7: {
                int n;
                cout << "Enter integer for factorial (0-20): ";
                cin >> n;
                if(n >= 0 && n <= 20) {
                    cout << "Factorial: " << factorial(n) << endl;
                } else {
                    cout << "Invalid range for recursive factorial!"
<< endl;
                }
                break;
            }
            case 8: {
                int n;
                cout << "Enter number of Fibonacci terms: ";
                cin >> n;
                fibonacci(n);
                break;
            }
            case 9: {
                double a, b, mx, mn;
                cout << "Enter two numbers: ";
                cin >> a >> b;
                findMaxMin(a, b, mx, mn);
                cout << "Maximum: " << mx << ", Minimum: " << mn <<
endl;
                break;
            }
            case 10:
                cout << "Exiting IT Study Hub Tool Suite. Goodbye!"
<< endl;
                break;
            default:
                cout << "Invalid choice! Please select between 1 and
10." << endl;
        }
        cout << "\n----------------------------------------\n";
    } while(choice != 10);

    return 0;
}

// Function Definitions
void showMenu() {
    cout << "========================================\n";
    cout << "    IT STUDY HUB NUMBER TOOL SUITE      \n";
```

```
    cout << "=======================================\n";
    cout << "1. Addition\n2. Subtraction\n3. Multiplication\n4.
Division\n";
    cout << "5. Even/Odd Check\n6. Prime Check\n7. Factorial
(Recursive)\n";
    cout << "8. Fibonacci Series\n9. Max/Min Comparison (Pass by
Ref)\n10. Exit\n";
    cout << "Enter your choice (1-10): ";
}

double add(double a, double b) { return a + b; }
double subtract(double a, double b) { return a - b; }
double multiply(double a, double b) { return a * b; }
double divide(double a, double b, bool &success) {
    if(b == 0) {
        cout << "Error: Division by zero!" << endl;
        success = false;
        return 0;
    }
    return a / b;
}

bool isEven(int n) { return (n % 2 == 0); }

bool isPrime(int n) {
    if(n <= 1) return false;
    for(int i = 2; i * i <= n; i++) {
        if(n % i == 0) return false;
    }
    return true;
}

long long factorial(int n) {
    if(n <= 1) return 1;
    return n * factorial(n - 1);
}

void fibonacci(int n) {
    if(n <= 0) { cout << "Invalid term count." << endl; return; }
    long long t1 = 0, t2 = 1, nextTerm;
    cout << "Fibonacci Series: ";
    for(int i = 1; i <= n; i++) {
        if(i == 1) { cout << t1 << " "; continue; }
        if(i == 2) { cout << t2 << " "; continue; }
        nextTerm = t1 + t2;
        t1 = t2;
```

```
        t2 = nextTerm;
        cout << nextTerm << " ";
    }
    cout << endl;
}

void findMaxMin(double a, double b, double &maxOut, double &minOut)
{
    if(a > b) { maxOut = a; minOut = b; }
    else { maxOut = b; minOut = a; }
}
```

Expected Output:

```
========================================
    IT STUDY HUB NUMBER TOOL SUITE
========================================
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Even/Odd Check
6. Prime Check
7. Factorial (Recursive)
8. Fibonacci Series
9. Max/Min Comparison (Pass by Ref)
10. Exit
Enter your choice (1-10): 6
Enter a positive integer: 17
17 is Prime.
```

Step-by-Step Explanation:

- The program wraps execution inside a `do-while` loop controlled by user selection (`choice != 10`).
- A `switch` statement dispatches execution to dedicated modular functions for each operation.
- Pass-by-reference (`bool &success` and `double &maxOut, double &minOut`) allows functions to return status flags and multiple computed values back to `main()`.
- Recursive computation is utilized cleanly in the `factorial` function.

Practice Challenges:

- Add an option 11 to check whether a number is a Palindrome.
- Implement GCD (Greatest Common Divisor) calculation using recursion.

IT STUDY HUB

Module Assessments

Test your mastery of control flow and functions with these rigorously curated questions.

15+ Practice Questions

1. What is the difference between an `if-else` ladder and a `switch` statement?
2. Write a program that uses a `for` loop to compute the sum of the first 50 natural numbers.
3. Explain why a `do-while` loop is guaranteed to execute its body at least once.
4. What is the exact output of `break` versus `continue` inside a `while` loop?
5. Write a function prototype for a function that accepts a string and an integer, returning a boolean.
6. Explain function overloading rules and give an example of two overloaded functions.
7. What is recursion? What are the two mandatory components every recursive function must possess?
8. What is the difference between pass-by-value and pass-by-reference parameters?
9. Why is `const` modifier useful when passing large objects by reference?
10. What is an inline function, and what compiler trade-off does it involve?
11. Write a `while` loop that reverses the digits of an integer.
12. Explain variable shadowing in nested scopes.
13. What happens if a recursive function lacks a base case?
14. Write a program using nested `for` loops to print a right-angled triangle of asterisks.
15. What is the purpose of function declarations (prototypes) placed before `main()`?

20+ Viva Questions (With Answers)

Q1: What is control flow?

> The order in which individual statements, instructions, or function calls are executed or evaluated in a program.

Q2: What is the primary difference between `if` and `switch`?

> `if` evaluates boolean range conditions; `switch` evaluates discrete constant values for exact equality.

Q3: Can a `switch` statement evaluate floating-point variables?

> No, `switch` expressions must evaluate to integral types or characters.

Q4: What is fall-through in a `switch` statement?

> When execution continues into subsequent cases because a `break` statement was omitted.

Q5: What are the three components of a `for` loop header?

> Initialization, condition check, and increment/update expression.

Q6: What is an infinite loop?

> A loop whose termination condition never evaluates to false, causing it to run indefinitely.

Q7: What does the `break` keyword do?

> Immediately terminates the enclosing loop or switch statement.

Q8: What does the `continue` keyword do?

> Skips the remaining statements in the current loop iteration and jumps to the next iteration.

Q9: What is a function prototype?

> A declaration of a function's name, return type, and parameters placed before its definition.

Q10: What is the return type of a function that does not return any value?

> `void`.

Q11: What are default arguments?

> Parameters assigned default values in the function signature, allowing callers to omit them.

Q12: What is function overloading?

> Defining multiple functions with the same name but distinct parameter signatures.

Q13: What is recursion?

> A programming technique where a function calls itself to solve smaller sub-problems.

Q14: What is a base case in recursion?

> The terminating condition that stops recursive calls from repeating infinitely.

Q15: What is pass by value?

> Passing a copy of an argument into a function, leaving the original variable unchanged.

Q16: What is pass by reference?

> Passing the memory address reference of a variable, allowing the function to modify the original value.

Q17: What does the `const` keyword signify in a parameter list?

> It ensures the referenced object cannot be modified inside the function.

Q18: What is an inline function?

> A function whose body is expanded directly at each call site to eliminate call overhead.

Q19: What is scope?

> The region of a program where a declared identifier is accessible.

Q20: Why is `goto` discouraged?

> It creates unstructured spaghetti code that is extremely difficult to debug and maintain.

15+ Technical Interview Questions

1. **Explain how the compiler evaluates `switch` statements versus `if-else` ladders under the hood.** Discuss jump tables (`switch`) versus sequential branch instructions (`if-else`).
2. **What is tail recursion, and why is it important for compiler optimization?** Explain how tail-call optimization (TCO) reuses stack frames to prevent stack overflow.
3. **When should you pass parameters by reference versus by pointer versus by value?** Compare semantics, nullability, and syntax.

4. **What are the dangers of default arguments in overloaded functions?** Discuss ambiguity resolution errors when multiple overloads match caller arguments.
5. **Explain how local variables are stored on the call stack during function invocation.** Walk through stack frame allocation and deallocation upon return.
6. **What is the exact mechanism of `inline` functions? Can a recursive function be inline?** Explain why recursion cannot truly be inlined because depth is unknown at compile time.
7. **How do you write a thread-safe recursive algorithm?** Discuss avoiding mutable static state inside recursive functions.
8. **What is copy-on-write and how does pass-by-reference-to-const optimize string handling?** Explain elimination of temporary object duplication.
9. **Can an `inline` function be defined inside a `.cpp` file instead of a header? Why or why not?** Discuss linkage and why the compiler needs visibility of the definition at every call site.
10. **What is variable shadowing, and how does scope resolution operator `::` access shadowed globals?** Provide code demonstrating global variable retrieval.
11. **Why does passing heavy objects by value severely impact application performance?** Discuss CPU cache thrashing and constructor/destructor overhead.
12. **How do you implement a non-recursive (iterative) version of a recursive algorithm like Fibonacci?** Compare time and space complexity ($O(N)$ time, $O(1)$ space).
13. **What is the significance of function linkage in C++ (`extern "C"`)?** Explain name mangling in C++ and how it differs from C.
14. **How do you prevent infinite loops when handling invalid user input in a `cin` stream?** Discuss stream error state flags (`cin.fail()`) and buffer clearing (`cin.clear()`, `cin.ignore()`).
15. **What are function objects (functors) and how do they compare to inline functions? (Preview)** Discuss stateful function execution.

Module 3 Wrap-Up

Key Terms Glossary

- **Branching:** Conditional execution paths (`if`, `switch`).
- **Iteration:** Repeated execution loops (`for`, `while`, `do-while`).
- **Jump Statement:** Flow alteration statements (`break`, `continue`, `goto`).
- **Prototype:** Function signature declaration prior to definition.
- **Overloading:** Multiple functions sharing a name with distinct parameter lists.
- **Recursion:** A function calling itself to solve sub-problems.
- **Pass by Reference:** Passing memory references (`&`) to modify originals.

Quick Revision Summary

Module 3 establishes control flow and modular programming. Conditions (`if`, `else-if`, `else`, `switch`) enable branching logic. Loops (`for`, `while`, `do-while`) automate repetition, managed by `break` and `continue`. Functions encapsulate logic through signatures, parameters, and return values. Advanced features include default arguments, function overloading, inline expansion, recursion, and pass-by-reference mechanics.

Beginner Coding Challenges

Complete these challenges before moving to Module 4:

- **Challenge 1: The Guessing Game.** Write a program where the computer generates a secret number, and the user guesses repeatedly using `while` loops with 'Too High' or 'Too Low' feedback.
- **Challenge 2: Recursive Power Calculator.** Write a recursive function `power(base, exp)` that computes exponentiation without using loops.
- **Challenge 3: Reference Swapper.** Write a function that takes three integer references (`a`, `b`, `c`) and rotates their values in place (`a` gets `b`, `b` gets `c`, `c` gets `a`).

CONGRATULATIONS ON COMPLETING MODULE 3!

WHAT COMES NEXT:

MODULE 4 — ARRAYS, STRINGS & POINTER BASICS